Real-Time Multi-Object Tracking

with Custom Data Association and Motion Prediction

CS 5330 — Pattern Recognition and Computer Vision

Kushwanth Vellanki • Mayank Goud Polagoni

Spring 2026



Northeastern University



Project Overview

Northeastern University

What We Built

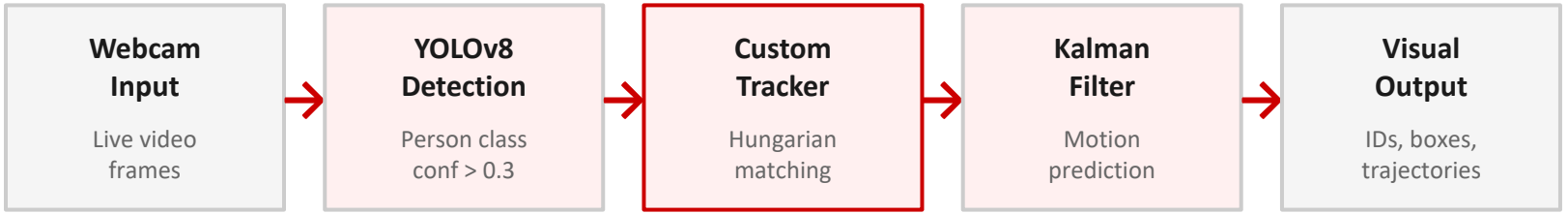
- Real-time tracking system using webcam feed
- YOLOv8n for object detection (person class)
- Custom tracker — not using SORT or DeepSORT
- Kalman Filter (4D state, 2D measurement) for motion prediction
- Hungarian algorithm for detection-to-track matching

Goals

- Track multiple objects with consistent IDs
- Predict short-term motion of each object
- Compare IOU vs distance-based matching
- Evaluate tracking under occlusion & speed variations
- Measure real-time performance (FPS)

Main Contribution: Custom tracking and data association logic built from scratch — understanding the internals instead of relying on off-the-shelf trackers.

System Pipeline



Key Implementation Details

Detection	YOLOv8n pretrained model — filters for person class (class_id=0) with confidence threshold 0.3
Tracking	Custom MultiObjectTracker class — maintains track list, creates/deletes tracks based on missed frames (max_missed=20)
Matching	Cost matrix built with IOU or Euclidean distance — solved using scipy.optimize.linear_sum_assignment (Hungarian)
Prediction	OpenCV Kalman Filter — 4D state [cx, cy, vx, vy], 2D measurement [cx, cy], trajectory history capped at 30 frames

Experiment 1 — Single vs Multiple Objects



Single person — ID 0, stable tracking



Two persons — ID 0 & ID 1, both stable

Single Person

- Tracking was stable with consistent ID assignment
- When person moved, tracking followed smoothly and ID remained the same

Multiple Persons

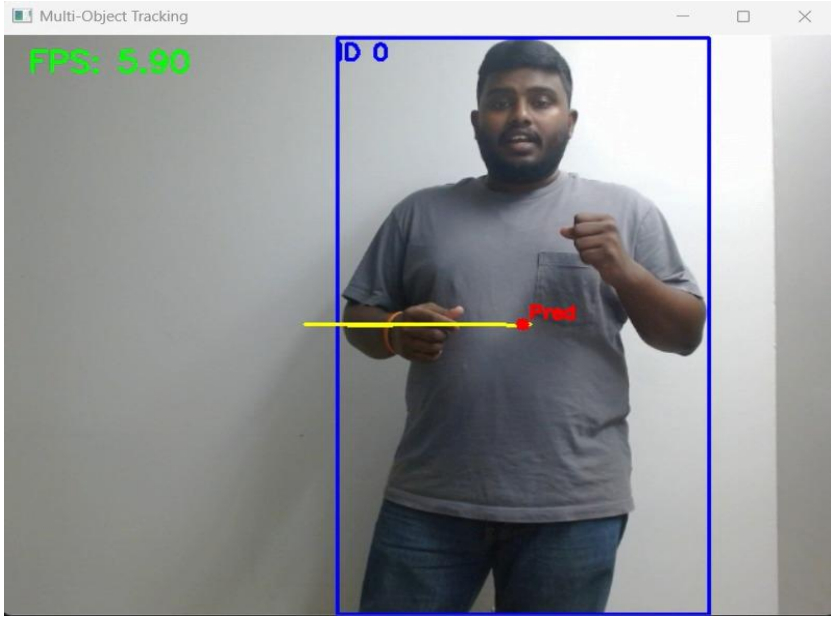
- Both persons were given different IDs — tracking was stable for both
- During movement, IDs remained consistent and did not swap

Re-entry Behavior

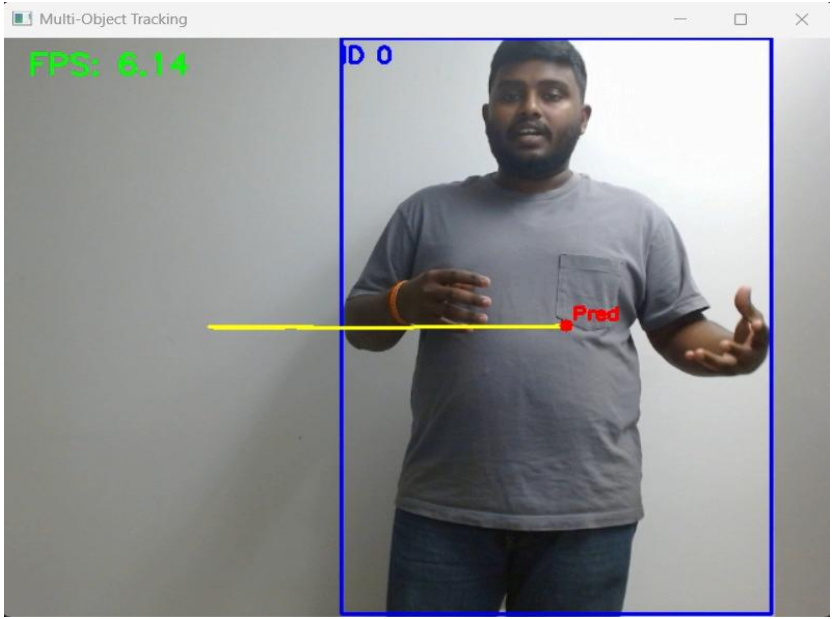
- When a person left the frame and came back, a new ID was assigned
- This is expected — no re-identification module was implemented

System tracks single and multiple objects well with stable IDs. Re-entry results in a new ID since no re-identification is implemented.

Experiment 2 — Slow vs Fast Motion



Slow movement — short yellow trajectory line



Fast movement — longer yellow trajectory trail

Observations

- Slow motion: yellow tracking line is short, ID is stable, tracking is smooth
- Fast motion: yellow tracking line is longer (more movement history), ID still stable, tracking remains good
- Kalman filter adapts well to both speeds — prediction accurate in both cases

Experiment 3 — IOU vs Distance-Based Matching

```
tracker = MultiObjectTracker(
    max_missed=20,
    iou_threshold=0.3,
    distance_threshold=120,
    use_iou=True
)
```

use_iou = True (IOU mode)

```
tracker = MultiObjectTracker(
    max_missed=20,
    iou_threshold=0.3,
    distance_threshold=120,
    use_iou=False
)
```

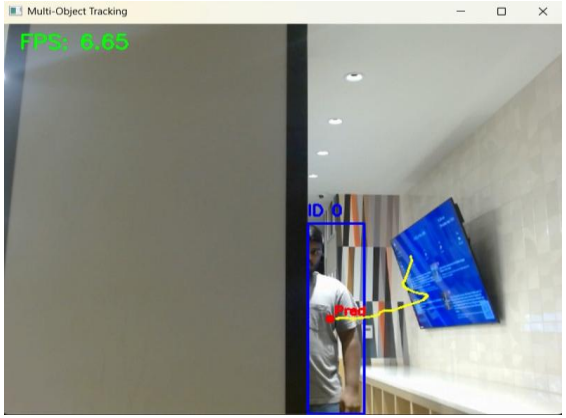
use_iou = False (Distance mode)

Hungarian Algorithm
Cost matrix: rows = tracks, cols = detections. Solved via scipy linear_sum_assignment. Matches below threshold are rejected.

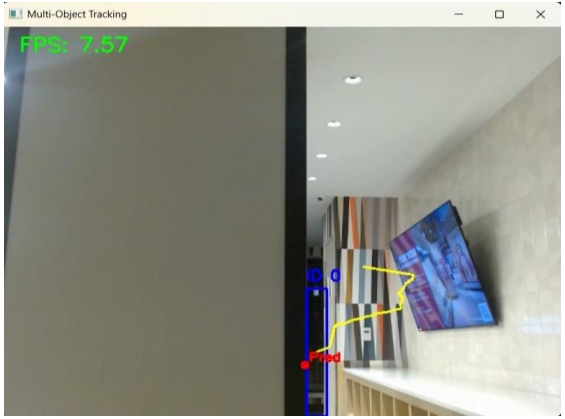
Metric	IOU (use_iou=True)	Distance (use_iou=False)
Tracking Stability	Stable — IDs consistent	Stable — IDs consistent
Speed	Faster	Somewhat slower
Cost Function	1 − IoU(track_bbox, det_bbox)	Euclidean distance between centers
Threshold	iou_threshold = 0.3	distance_threshold = 120 px

Both methods produce stable tracking. IOU mode is faster; distance mode is somewhat slower but both work well for our use case.

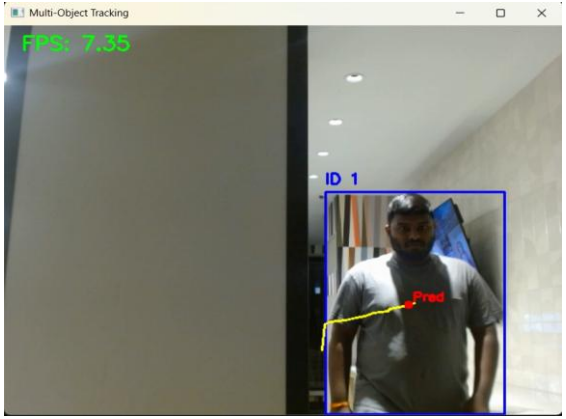
Experiment 4 — Occlusion Handling



Partial occlusion — ID 0



Out of frame — leaving view



Re-entry — assigned new ID 1

Partial Occlusion

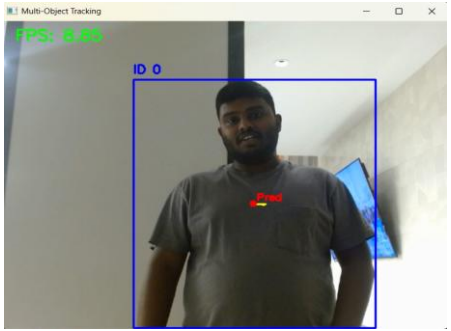
- Body was partially covered — tracker used the visible part to continue tracking
- ID remained stable throughout the occlusion

Full Occlusion / Out of Frame

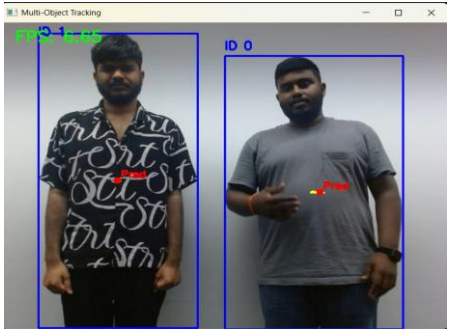
- Person went completely out of frame — track was deleted after max_missed=20 frames
- On re-entry, a new ID was assigned (ID 0 → ID 1)

Partial occlusion handled well — tracking uses visible body parts. Full occlusion treated as exit; new ID on re-entry (no re-identification).

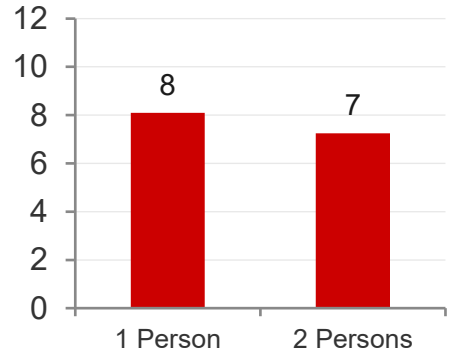
Experiment 5 — FPS Analysis & Conclusion



1 person: ~7.4–8.8 FPS



2 persons: ~6.5–8.0 FPS



FPS drops slightly with more tracked objects, but system remains real-time.

What We Achieved

- Complete detection + tracking + prediction pipeline from scratch
- Custom Hungarian matching (IOU & distance modes)
- Stable multi-object tracking with consistent IDs
- Real-time Kalman filter prediction with trajectory visualization

Limitations

- No re-identification after object leaves frame
- Only tested on person class

Future Work

- Add appearance-based re-identification
- Optimize for higher FPS (GPU, quantization)
- Handle complex multi-class scenes